38. Breakpoint'ler, Loglar ve Debugger Araçlarını Efektif Kullanmak

'console.log' Kirliliğinden Kurtulup Zamanı Durdurma Sanatı
Bölüm 7 • Zorluk Derecesi: Orta Seviye • Tahmini Okuma Süresi: 12 dk

---

> 🎙️ **NotebookLM & Audio Overview Podcast Notları**
>
> **Podcast Tartışma Sorusu:**
> *"Yazılımcılar neden koda 50 tane `console.log('buraya geldi 1')` yazmayı alışkanlık haline getirir ve interaktif debugger kullanmak bu süreyi nasıl 10 kat kısaltır?"*
>
> **Bölümden Çıkarılacak 3 Altın Cümle:**
> - 📌 `console.log` ile hata aramak karanlık bir odada el feneriyle küçük bir noktaya bakmaktır; Debugger ise odanın ana ışığını yakmaktır. - 📌 Koşullu Breakpoint (Conditional Breakpoint): Sadece belirli bir koşul sağlandığında (örn: `i === 5000` veya `user.id === 'target'`) durur.
> - 📌 Call Stack (Çağrı Yığını) penceresi, o fonksiyonun hangi fonksiyonlar tarafından çağrıldığını zamanda geriye giderek görmenizi sağlar.

📖 1. Giriş Hikayesi & Günlük Hayat Analojisi

Bir futbol maçını televizyonda izlediğinizi düşünün. Hakem ofsayt kararı verdi.

Eğer elinizde video durdurma teknolojisi olmasaydı ne yapardınız? Maçı baştan sona tekrar oynatıp, top ayağa değdiği anda gözünüzü kırpmadan ekrana bakmaya çalışırdınız! (İşte bu `console.log` ile kod çalıştırmaktır).

Peki VAR (Video Hakem) sistemi ne yapar? Tam topa vurulduğu milisaniyede görüntüyü **DURDURUR (Breakpoint)**. İleriye sarar (Step Over), geriye sarar, oyuncunun milimetrik pozisyonunu inceler (Scope Variables).

Modern IDE'lerin Debugger araçları sizin yazılımdaki VAR sisteminizdir. Zamanı dondurabilir, değişkenlerin içine girebilir ve hatayı nokta atışı yakalayabilirsiniz.

🧠 2. AI Çağında Yeni Zihniyet Modeli

Debugger Kontrol Tuşları Sözlüğü:

```
⏸️ PAUSE / RESUME (F5):
   Kodu bir sonraki breakpoint'e kadar normal hızında çalıştırır.

⏭️ STEP OVER (F10):
   Mevcut satırı çalıştırır ve alt satıra geçer (Fonksiyonun içine girmeden).

⬇️ STEP INTO (F11):
   Mevcut satırdaki fonksiyonun İÇİNE girer (İç mantığı satır satır görmek için).

⬆️ STEP OUT (Shift+F11):
   Bulunduğunuz fonksiyondan çıkar ve onu çağıran üst fonksiyona geri döner.

🔴 CONDITIONAL BREAKPOINT:
   Kodu sadece `user.age < 0` olduğunda durdurur (10.000 döngüyü tek tek gezmez!).
```

🤖 3. AI Prompt Laboratuvarı (Kopyala-Yapıştır Hazır Prompt)

Aşağıdaki prompt şablonunu Cursor, Claude Code, ChatGPT veya Copilot ortamınızda doğrudan kullanabilirsiniz:

```
# ROL: Debugger ve Geliştirme Araçları Mentoru
VS Code veya Cursor üzerinde Node.js uygulamamızı debug edeceğiz.

## GÖREV:
1. `.vscode/launch.json` dosyasını TypeScript ve `tsx` (veya ts-node) ile anında
breakpoint yakalayacak şekilde hazırla.
2. 10.000 elemanlı bir döngüde sadece belirli bir hata anında duracak 'Conditional
Breakpoint' kullanımını anlat.
3. Call Stack penceresini kullanarak hatanın hangi ara katmandan tetiklendiğini
nasıl tespit edeceğimi göster.
```

💻 4. Kod İncelemesi: Acemi / Halüsinasyon vs. Üretim Kalitesi

❌ **Acemi / Kontrolsüz AI Çıktısı (Riskli):**

```
// Kötü / İlkel Yöntem: launch.json yok, kodun her yerine console.log saçmak
// console.log("Burada 1");
// console.log("User:", user);
// console.log("Burada 2");
```

✅ **Kıdemli Mühendis Tarafından İyileştirilmiş Sürüm (Güvenli & Temiz):**

```
// Profesyonel VS Code / Cursor Debug Yapılandırması (.vscode/launch.json)
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "🐞 Debug Mevcut TypeScript Dosyası",
      "type": "node",
      "request": "launch",
      "runtimeExecutable": "npx",
      "runtimeArgs": ["tsx", "${file}"],
      "cwd": "${workspaceFolder}",
      "console": "integratedTerminal",
      "internalConsoleOptions": "neverOpen",
      "skipFiles": [
        "<node_internals>/**",
        "**/node_modules/**"
      ]
    },
    {
      "name": "🌐 Attach to Running Node Process",
      "type": "node",
      "request": "attach",
      "port": 9229,
      "restart": true,
      "skipFiles": ["<node_internals>/**"]
    }
  ]
}
```

💡 **Kod Analizi:** `skipFiles` ayarı sayesinde debug esnasında yanlışlıkla `node_modules` veya Node.js dahili kütüphanelerine düşmezsiniz; debugger sadece sizin kaynak kodunuz üzerinde adım adım ilerler.

⚠️ 5. Junior Tuzakları & Kıdemli Bakış Açısı

- **Tuzak:** Koda geçici `console.log` yazıp production'a commit etmek ve logları kirletmek.
  - **Çözüm:** Her zaman interaktif breakpoint kullanın; commit öncesi `git diff` ile log kalmadığını teyit edin.

- **Tuzak:** Büyük döngülerde normal breakpoint koyup 10.000 kez F5'e basmaya çalışmak.
  - **Çözüm:** Koşullu Breakpoint (Conditional Breakpoint) koyarak sadece `id === 'hatali-kayit'` anında durmasını sağlayın.
- **Tuzak:** Asenkron fonksiyonlarda `await` satırında Step Over yaparken thread'in atlamasını anlayamamak.
  - **Çözüm:** Event Loop mantığını hatırlayın; asenkron çağrılarda Step Into yerine bir sonraki satıra breakpoint koyun.

🎬 6. Udemy Video Dersi Taslağı & Slayt Maddeleri

📽️ Ders Başlığı: *Ders 38: Debugger Ustalığı - console.log'a Veda Etmek*

- 🖥️ **Slayt 1:** Neden console.log ile hata aramak zaman kaybıdır?
- 🖥️ **Slayt 2:** VS Code ve Cursor Debugger anatomisi: F5, F10, F11.
- 🖥️ **Slayt 3:** Koşullu Breakpoint (Conditional Breakpoints) ve Logpoints.
- 🖥️ **Slayt 4:** Canlı Demo: launch.json ile çalışan backend'e bağlanıp zamanı durdurmak.

Canlı Kodlama / Ekran Kaydı Senaryosu:
VS Code'da bir fonksiyona kırmızı breakpoint koyun. F5 ile debug modunu başlatın. Kod durduğunda sol paneldeki 'Variables' alanını açın; fonksiyonun içindeki tüm değişkenlerin canlı değerlerini, scope'unu ve Call Stack'i gösterin.

🎯 7. Günün Uygulamalı Görevi & Mini Sınav

🛠️ Günün Görevi:

Projenizde `.vscode/launch.json` dosyasını oluşturun ve bir sonraki hatada `console.log` yazmak yerine kırmızı bir breakpoint koyarak değişkenleri inceleyin.

🧠 Bölüm Sonu Kontrol Testi:

Soru 1: Bir döngü içinde 5.000 eleman varken, sadece `i === 4250` olduğunda kodun durmasını sağlamak için hangi breakpoint türü kullanılmalıdır? - A) Standart Breakpoint - B) Koşullu Breakpoint (Conditional Breakpoint) - C) Logpoint - D) CSS Breakpoint > **Doğru Yanıt:** B > **Açıklama:** Koşullu breakpoint, içine yazılan mantıksal ifade true olana kadar kodu durdurmaz; böylece binlerce gereksiz adımı atlar.

Soru 2: Debugger panelinde 'Call Stack' (Çağrı Yığını) penceresinin geliştiriciye sunduğu en önemli bilgi nedir? - A) Bilgisayarın pil yüzdesi - B) O an durulan fonksiyonun hangi fonksiyonlar zinciri tarafından çağrıldığını (kimin kimi tetiklediğini) zamanda geriye dönük göstermesi - C) Kodun satır sayısı -

D) GitHub şifresi > **Doğru Yanıt:** B > **Açıklama:** Call Stack, o anki yürütme noktasına ulaşana kadar hangi fonksiyonların sırayla çağrıldığını gösteren geçmiş izidir.